



中国科学技术大学
University of Science and Technology of China

语法制导的翻译 I

《编译原理(H)》

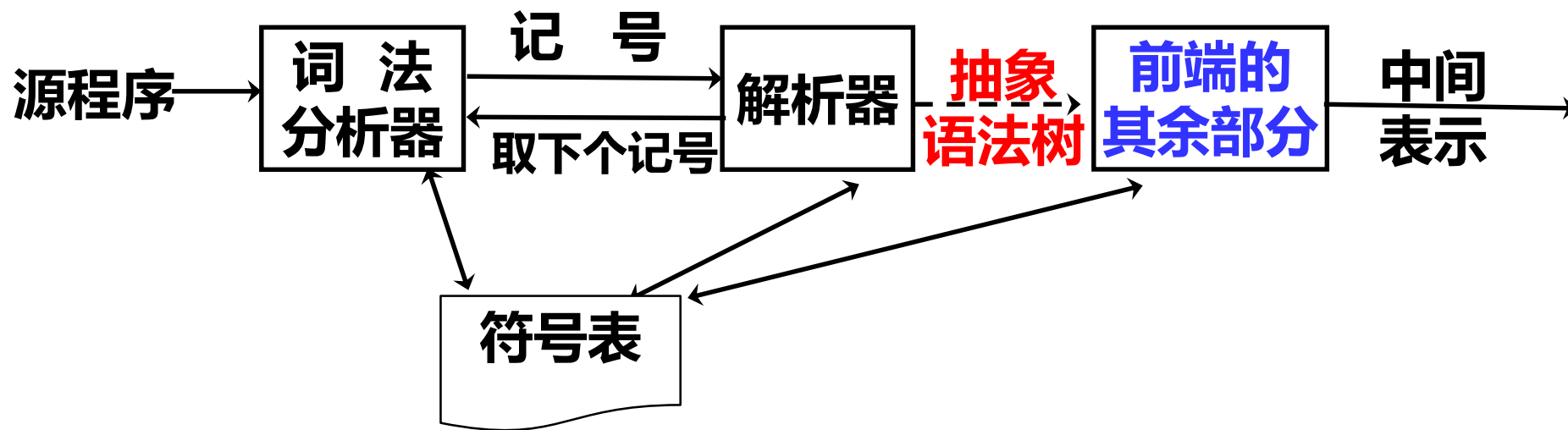
张昱

0551-63603804, yuzhang@ustc.edu.cn

中国科学技术大学
计算机科学与技术学院



本章内容



□ 语义的描述：语法制导的定义、翻译方案

- 语法制导：syntax-directed
按语法结构来指导语义的定义和计算
- 抽象语法树、注释解析树等

□ 语法制导翻译的实现方法：自上而下、自下而上

- 边语法分析边翻译



4.1 语法制导的定义

- 语法制导的定义
- 综合属性、继承属性
- 属性依赖图与属性的计算次序
- S属性定义、L属性定义



语法制导的定义

□ 语法制导的定义(Syntax-Directed Definition)

■ 基础的上下文无关文法

■ 每个文法符号有一组属性

用来表示语法成分
对应的语义信息

描述语义属性值
的计算规则

■ 每个文法产生式 $A \rightarrow \alpha$ 有一组形式为 $b = f(c_1, c_2, \dots, c_k)$ 的语义规则

其中 f 是函数， b 和 $c_1, c_2, \dots, c_k$ 是该产生式文法符号的属性

✓ b 是综合属性(synthesized attribute): 如果 b 是 A 的属性， $c_1, c_2, \dots, c_k$ 是产生式右部文法符号的属性或 A 的其它属性

—— 由子结点计算得来，自下而上传递

✓ b 是继承属性(inherited attribute): 如果 b 是产生式右部某个文法符号 X 的属性， $c_1, c_2, \dots, c_k$ 是 A 的属性或右部文法符号的属性

—— 由兄弟结点、父结点和自己的属性值来计算，同层或自上而上传递



例题 1

下面是产生字母表 $\Sigma = \{0, 1, 2\}$ 上数字串的一个文法：

$$S \rightarrow D S D \mid 2$$

$$D \rightarrow 0 \mid 1$$

写一个语法制导定义，判断它接受的句子是否为回文数

$$S' \rightarrow S$$

`print(S.val)`

$$S \rightarrow D_1 S_1 D_2$$

`S.val = (D1.val == D2.val) and S1.val`

$$S \rightarrow 2$$

`S.val = true`

$$D \rightarrow 0$$

`D.val = 0`

$$D \rightarrow 1$$

`D.val = 1`

对 S 和 D 加下标
以区分同类语法
结构的不同实例

$S.val$: S 对应的串是否是回文数

$D.val$: D 对应的串的数值

各文法符号的属性均是**综合属性**的语法制导定义—— **S 属性定义**



简单计算器的语法制导定义

产生式	语义规则	L 的匿名属性
$L \rightarrow E \textcolor{blue}{n}$ 换行标记	$\text{print}(E.val)$	
$\textcolor{red}{E} \rightarrow \textcolor{red}{E}_1 + T$	$\textcolor{red}{E}.val = \textcolor{red}{E}_1.val + T.val$	
$E \rightarrow T$	$E.val = T.val$	
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$	
$T \rightarrow F$	$T.val = F.val$	
$F \rightarrow (E)$	$F.val = E.val$	
$F \rightarrow \text{digit}$	$F.val = \text{digit}.\textcolor{blue}{lexval}$ 由词法分析给出	

各文法符号的属性均是**综合属性**的语法制导定义——**S 属性定义**

参见: bison-examples.tar.gz 中的 `config/expr1.y`, `expr.lex`

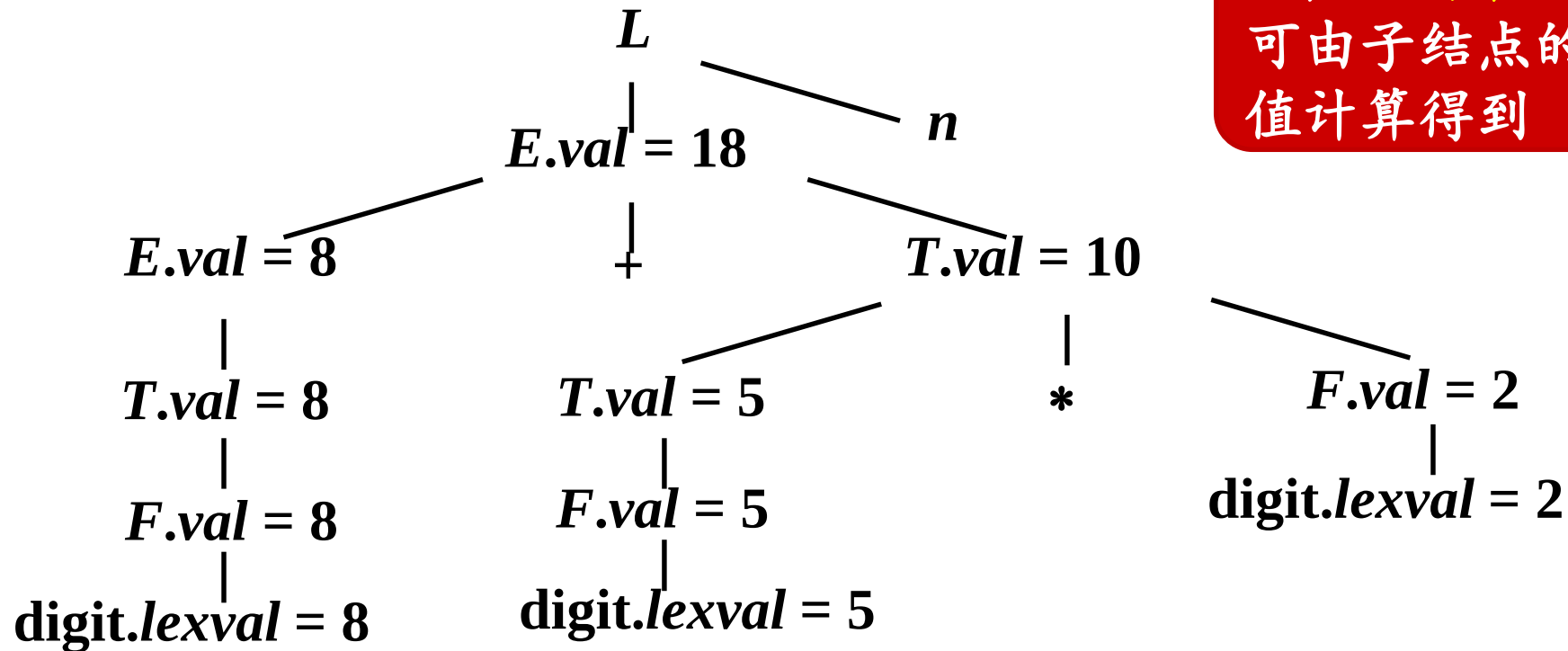
张昱: 语法制导的翻译



注释解析树 (annotated parse tree)

□ 结点的属性值都标注出来的解析树

$8+5*2$ n (n为换行符)的注释解析树，在树的根结点打印18



结点的综合属性值
可由子结点的属性
值计算得到



继承属性示例

int id, id, id

产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow \text{int}$	$T.type = \text{integer}$
$T \rightarrow \text{real}$	$T.type = \text{real}$
$L \rightarrow L_1, \text{id}$	$L_1.in = L.in;$ $addType(id.entry, L.in)$
$L \rightarrow \text{id}$	$addType(id.entry, L.in)$

type – T 的综合属性

in – L 的继承属性，把声明的类型传递给标识符列表

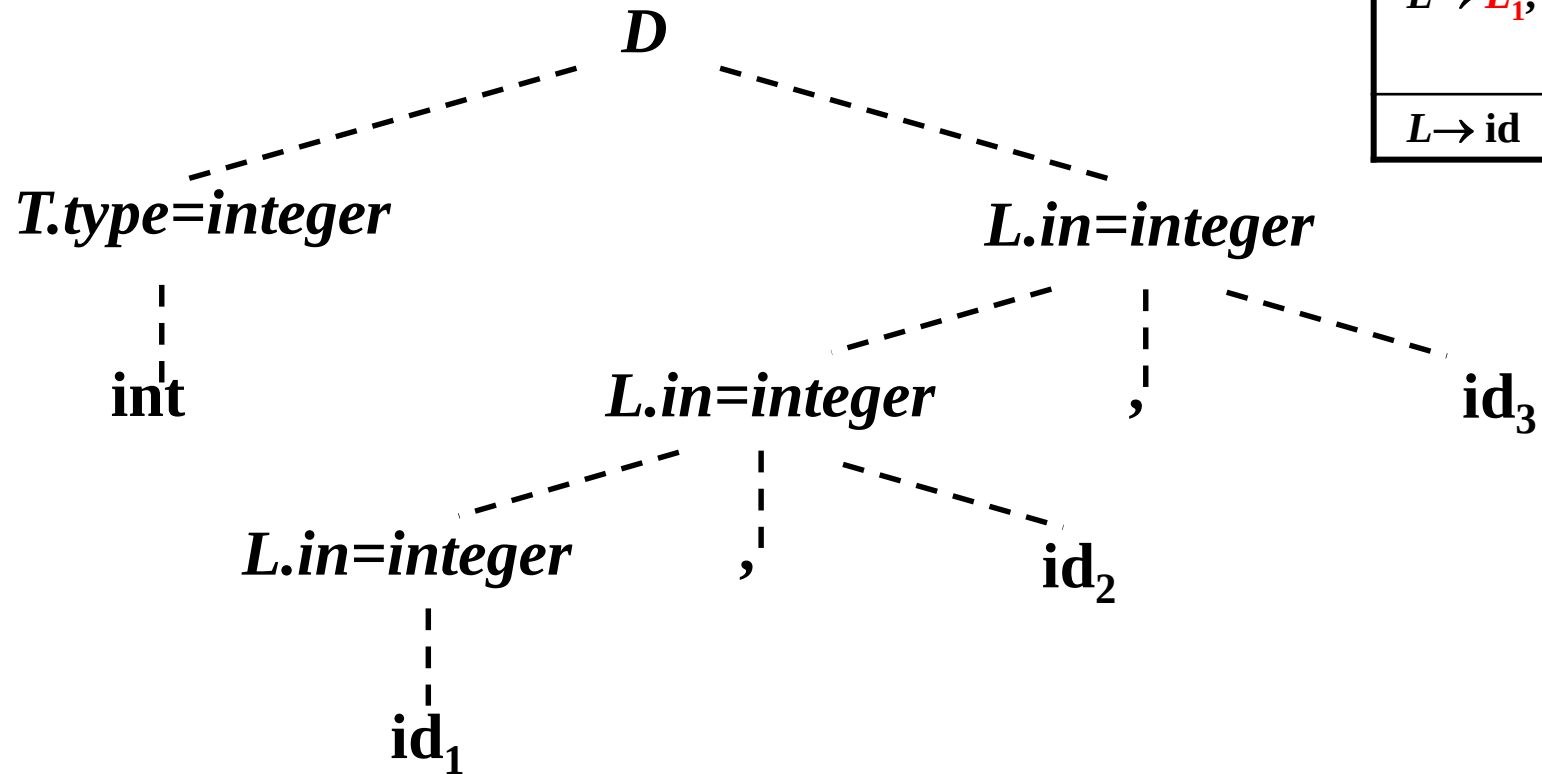
addType – 把类型信息加到符号表中的标识符条目里



注释解析树

`int id1, id2, id3`

解析树（虚线），注释了部分属性



产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow \text{int}$	$T.type = \text{integer}$
$T \rightarrow \text{real}$	$T.type = \text{real}$
$L \rightarrow L_1, \text{id}$	$L_1.in = L.in;$ $\text{addType}(\text{id.entry}, L.in)$
$L \rightarrow \text{id}$	$\text{addType}(\text{id.entry}, L.in)$

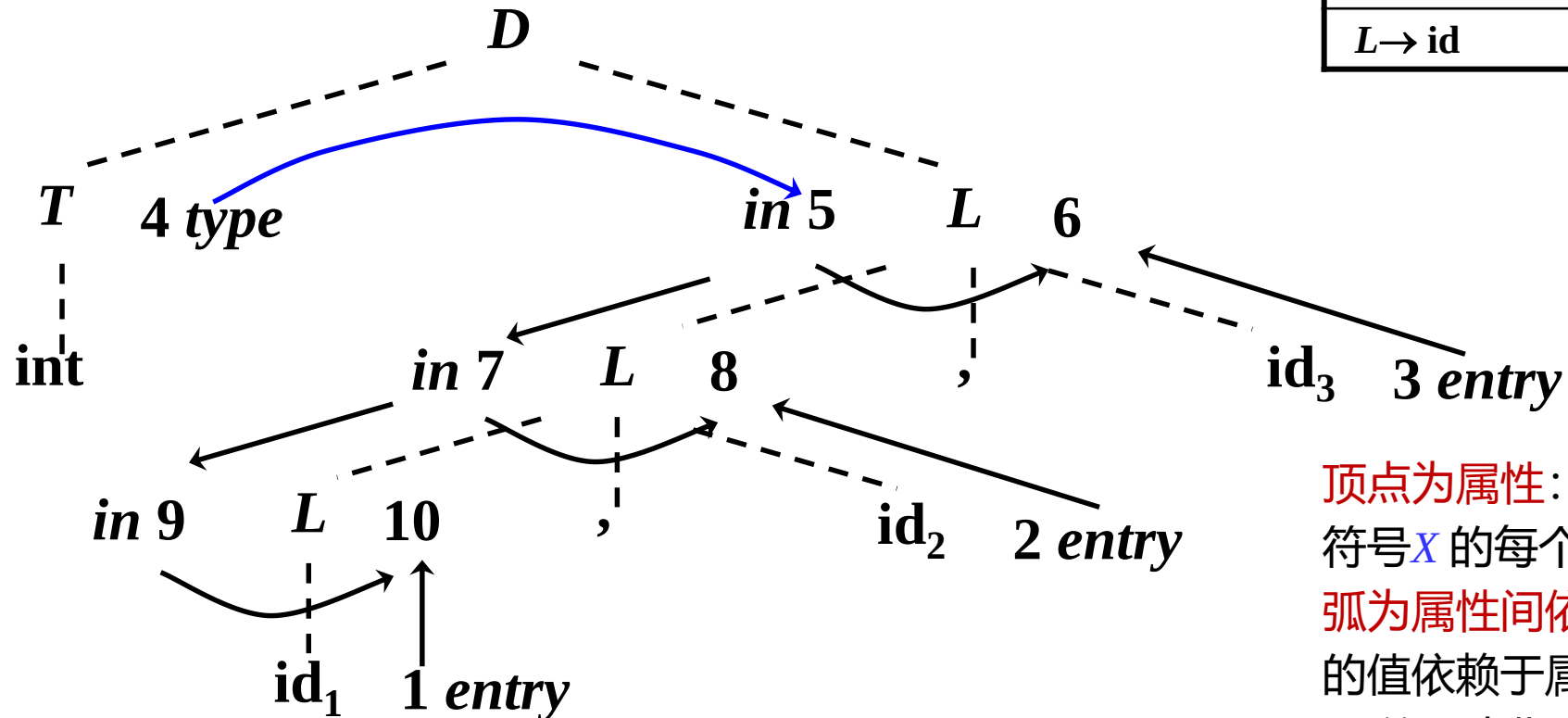


属性依赖图(dependence graph)

属性依赖图

描述解析树中结点的属性间依赖关系的有向图

int id₁, id₂, id₃的依赖图 (实线)



产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow \text{int}$	$T.type = \text{integer}$
$T \rightarrow \text{real}$	$T.type = \text{real}$
$L \rightarrow L_1, id$	$L_1.in = L.in;$ $addType(id.entry, L.in)$
$L \rightarrow id$	$addType(id.entry, L.in)$

顶点为属性：对应解析树中每个文法符号 X 的每个属性 a

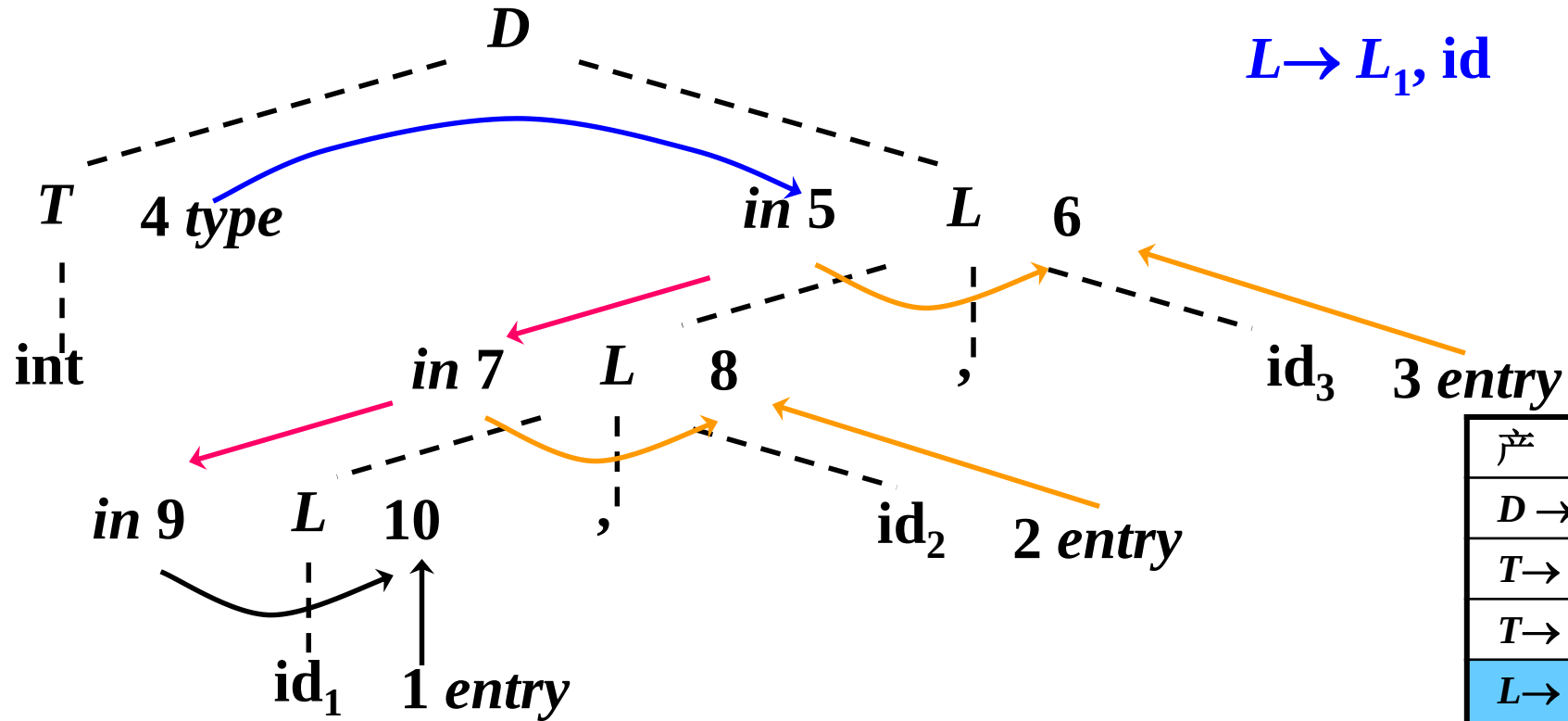
弧为属性间依赖关系：如果属性 $X.a$ 的值依赖于属性 $Y.b$ 的值，则存在从 $Y.b$ 的顶点指向 $X.a$ 的顶点的弧



属性依赖图(dependence graph)

int id₁, id₂, id₃

解析树 (虚线) 的依赖图 (实线)



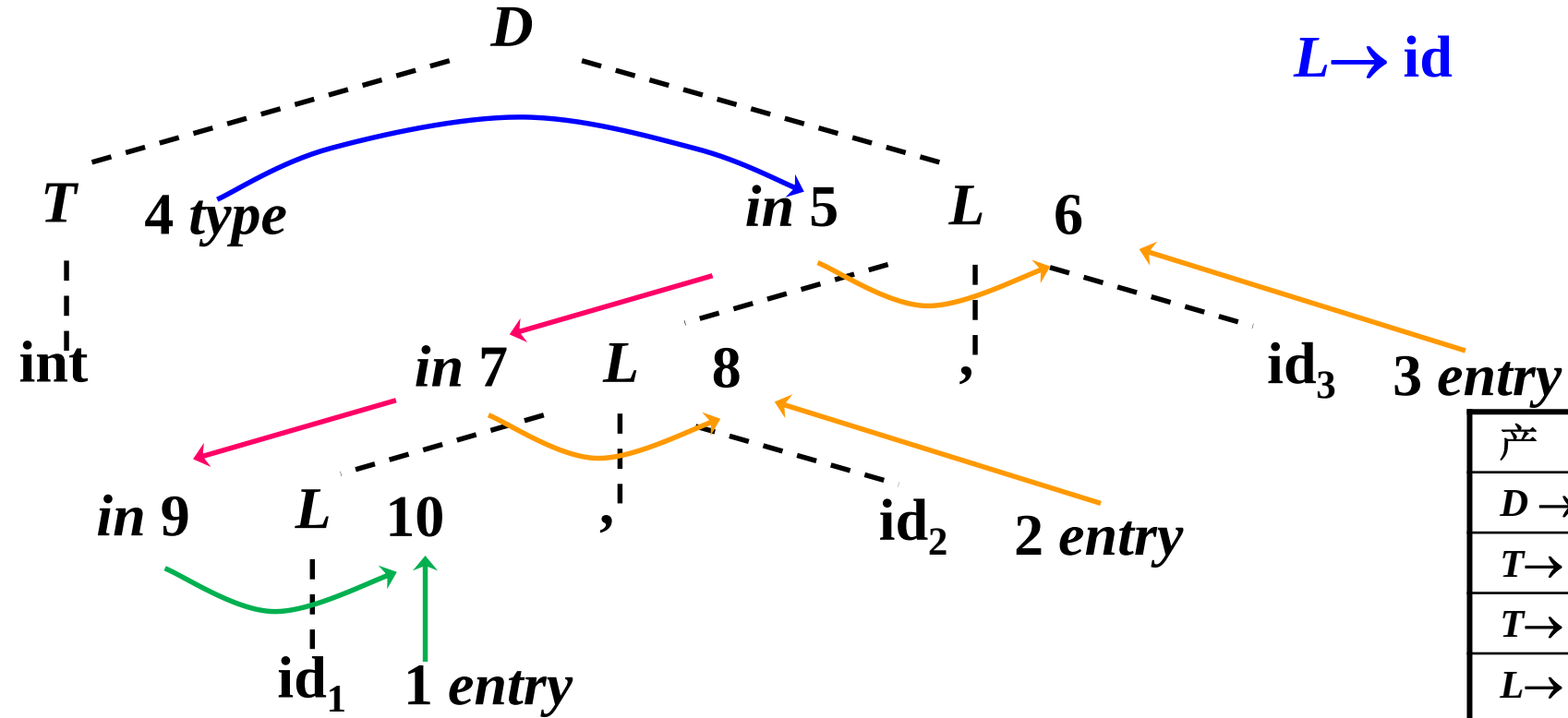
产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.in = L.in$; $addType(id.entry, L.in)$
$L \rightarrow id$	$addType(id.entry, L.in)$



属性依赖图(dependence graph)

int id_1, id_2, id_3

解析树 (虚线) 的依赖图 (实线)



$L \rightarrow id$

$addType(id.entry, L.in)$

产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.in = L.in;$ $addType(id.entry, L.in)$
$L \rightarrow id$	$addType(id.entry, L.in)$



S 属性定义和 L 属性定义

□ S 属性定义

仅含综合属性的语法制导定义

□ L 属性定义 (属性信息从左流向右)

如果每个产生式 $A \rightarrow X_1 \dots X_{j-1} X_j \dots X_n$ 的每条语义规则计算的属性是 A 的综合属性；或者是 X_j 的继承属性，但它仅依赖：

- 该产生式中 X_j 左边符号 $X_1, X_2, \dots, X_{j-1}$ 的属性；
- A 的继承属性

可以按边解析边翻译的方式计算继承属性

□ S 属性定义是 L 属性定义

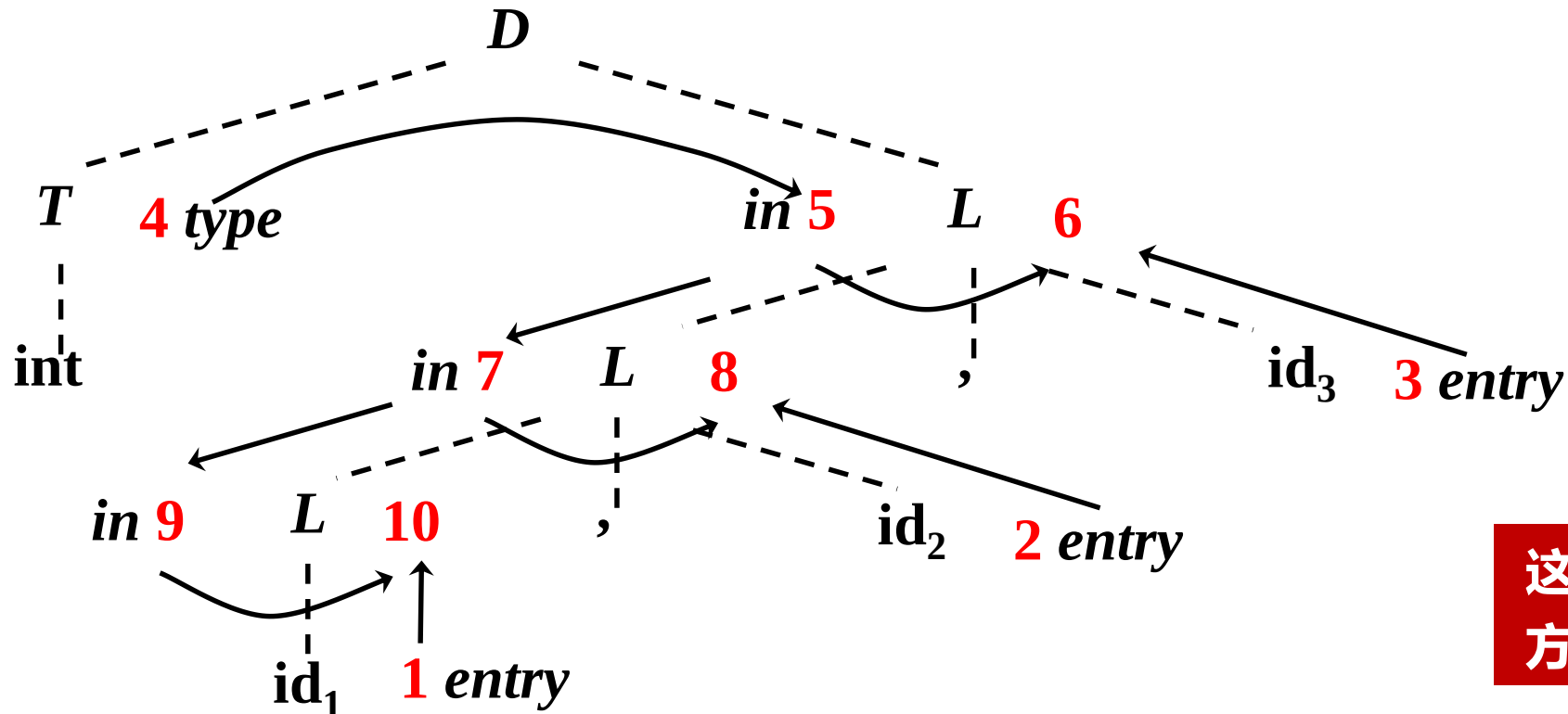
属性计算次序

- 拓扑排序(topological sort): 是DAG的结点的一种排序 $m_1, \dots, m_k$, 若有 m_i 到 m_j 的边, 则在排序中 m_i 先于 m_j

例 拓扑排序为结点1, 2, 3, 4, 5, 6, 7, 8, 9, 10

序号为 n 的结点的属性写成 a_n

```
a4 = integer;
a5 = a4;
addType(id3.entry, a5);
a7 = a5;
addType(id2.entry, a7);
a9 = a7;
addType(id1.entry, a9);
```



这种语义规则的计算
方法称为**解析树方法**



属性计算次序

■ 属性计算次序

- 1) 构造输入的解析树
- 2) 构造属性依赖图
- 3) 对结点进行拓扑排序
- 4) 按拓扑排序的次序计算属性



语义规则的计算方法

□ 解析树方法

刚才介绍的方法，动态确定**输入**的属性计算次序，效率低

——**概念上的一般方法**

□ 基于规则的方法

(编译器实现者) 静态确定 **(语言设计者提供的)** **语义规则**的计算次序

——**适用于手工构造的方法**

□ 忽略规则的方法

(编译器实现者) 事先确定属性的计算策略（如边解析边计算），**(语言设计者提供的)** 语义规则必须符合所选**解析策略**的限制

——**适用于自动生成的方法**



4.2 S属性定义的自下而上计算

- 语法树及其构造（文法对构造的影响）
 - 语法制导定义 vs. 翻译方案
- S属性定义的自下而上计算



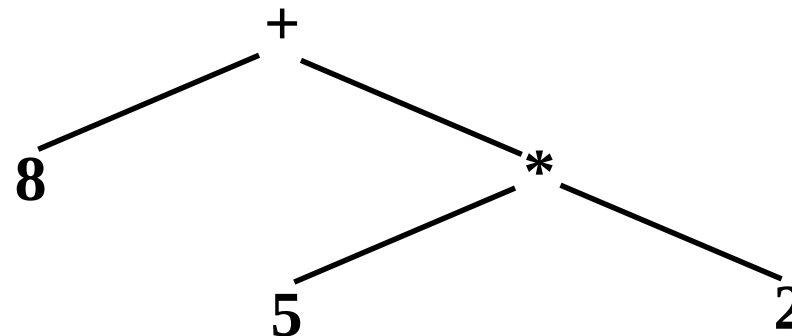
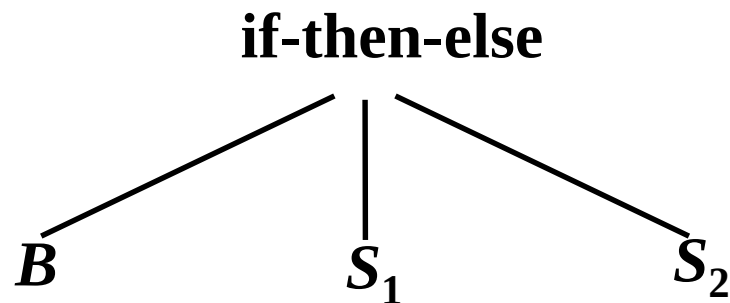
语法树(syntax tree)

□ 语法树是解析树的浓缩表示

每个结点表示一个语法构造，算符和关键字是语法树中的内部结点
举例：

if B then S_1 else S_2

$8 + 5 * 2$



语法制导翻译可以基于解析树，也可以基于语法树



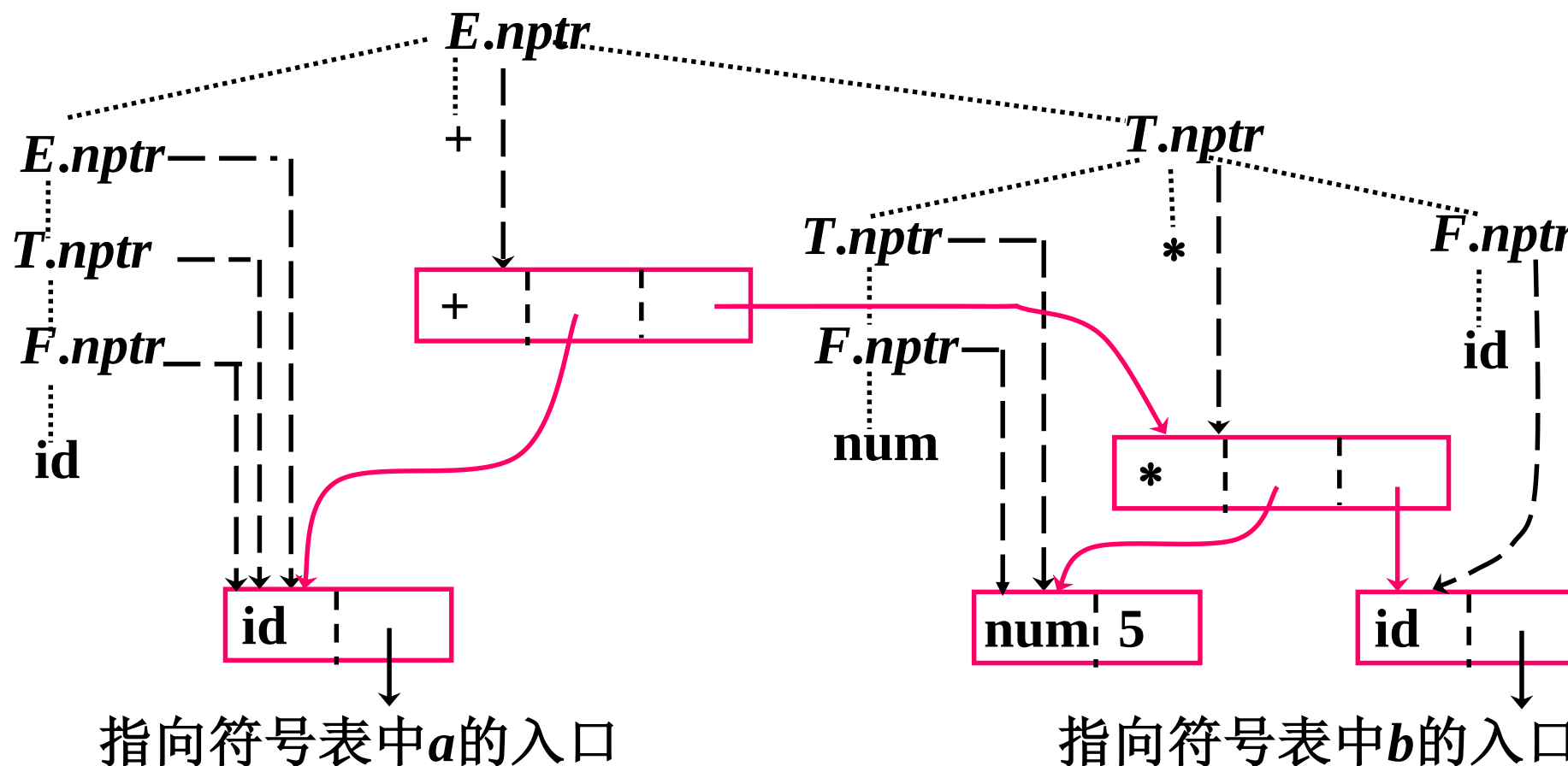
语法制导定义：构造语法树

产生式	语义规则
$E \rightarrow E_1 + T$	$E.nptr = mkNode('+', E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr = T.nptr$
$T \rightarrow T_1 * F$	$T.nptr = mkNode('*', T_1.nptr, F.nptr)$
$T \rightarrow F$	$T.nptr = F.nptr$
$F \rightarrow (E)$	$F.nptr = E.nptr$
$F \rightarrow id$	$F.nptr = mkLeaf(id, id.entry)$
$F \rightarrow num$	$F.nptr = mkLeaf(num, num.val)$

参见：bison-examples.tar.gz 中的 `config/asgn2ast.y`, `asgn.lex`



$a+5*b$ 的语法树的构造



实际执行的函数调用序列:

```

p1 = mkleaf(id, entrya);

```

```
p2 = mkleaf(num, 5);
```

```

p3 = mkleaf(id, entryb);

```

```
p4 = mknode('*', p2, p3);
```

```

p5 = mknode('+', p1, p4);

```



翻译方案

□ 构造语法树的翻译方案（左递归文法）

语义动作，
置于花括号中

$$\begin{aligned} E &\rightarrow E_1 + T && \{E.nptr = mkNode('+', E_1.nptr, T.nptr); \} \\ E &\rightarrow T && \{E.nptr = T.nptr; \} \\ T &\rightarrow T_1 * F && \{T.nptr = mkNode('*', T_1.nptr, F.nptr); \} \\ T &\rightarrow F && \{T.nptr = F.nptr; \} \\ F &\rightarrow (E) && \{F.nptr = E.nptr; \} \\ F &\rightarrow id && \{F.nptr = mkLeaf(id, id.entry); \} \\ F &\rightarrow num && \{F.nptr = mkLeaf(num, num.val); \} \end{aligned}$$

综合属性的计算规则置于产生式右部的右边，表示识别出右部后计算



4.2 S属性定义的自下而上计算

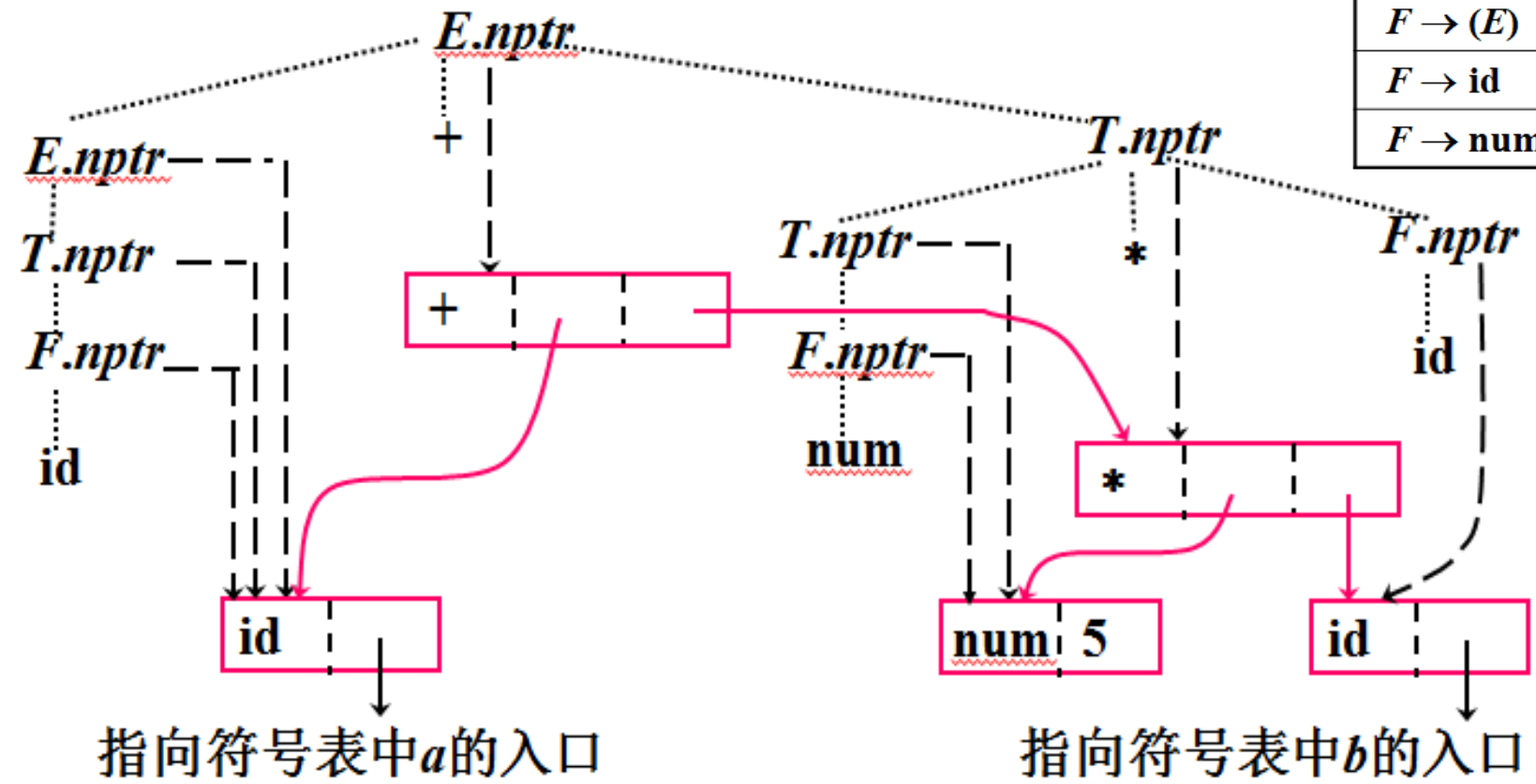
- 语法树及其构造（文法对构造的影响）
 - 语法制导定义 vs. 翻译方案
- S属性定义的自下而上计算



S属性定义举例

产生式	语义规则
$E \rightarrow E_1 + T$	$E.nptr = mkNode('+', E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr = T.nptr$
$T \rightarrow T_1 * F$	$T.nptr = mkNode('*', T_1.nptr, F.nptr)$
$T \rightarrow F$	$T.nptr = F.nptr$
$F \rightarrow (E)$	$F.nptr = E.nptr$
$F \rightarrow id$	$F.nptr = mkLeaf(id, id.entry)$
$F \rightarrow num$	$F.nptr = mkLeaf(num, num.val)$

$a+5*b$ 的语法树的构造

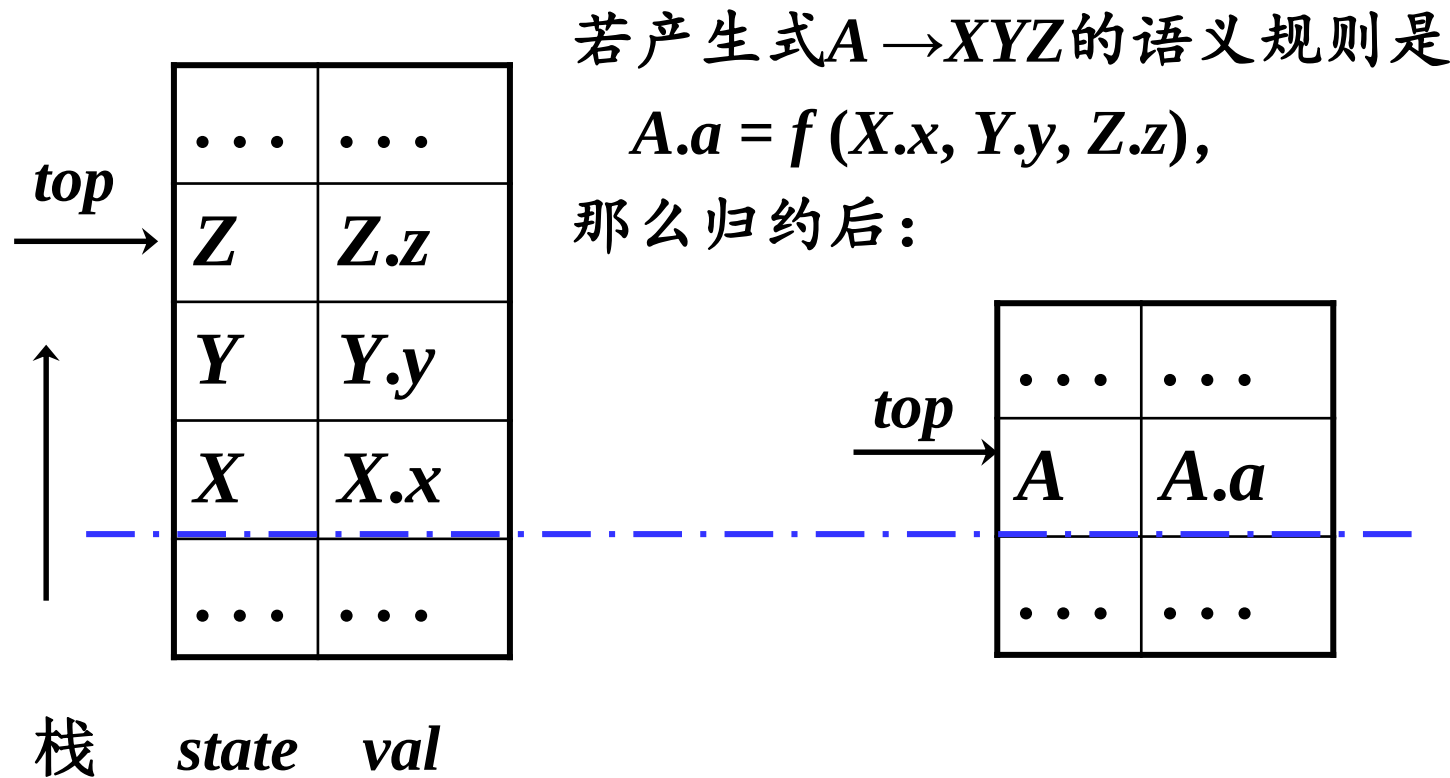




S属性的自下而上计算

□ 边解析边计算

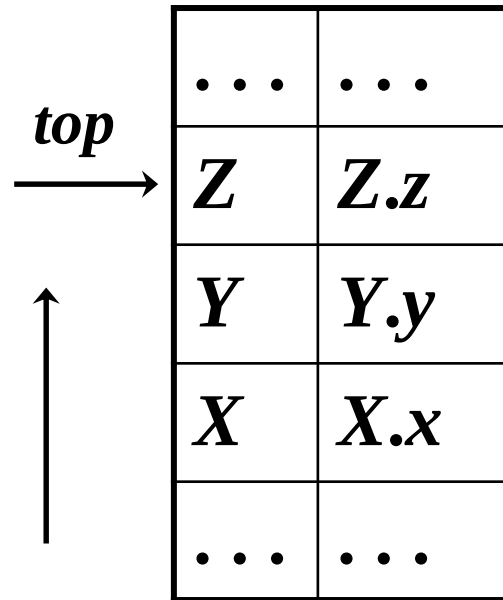
LR解析器的栈增加一个域来保存综合属性值





自下而上的翻译

例 简单计算器的语法制导定义改成栈操作代码



栈 *state* *val*

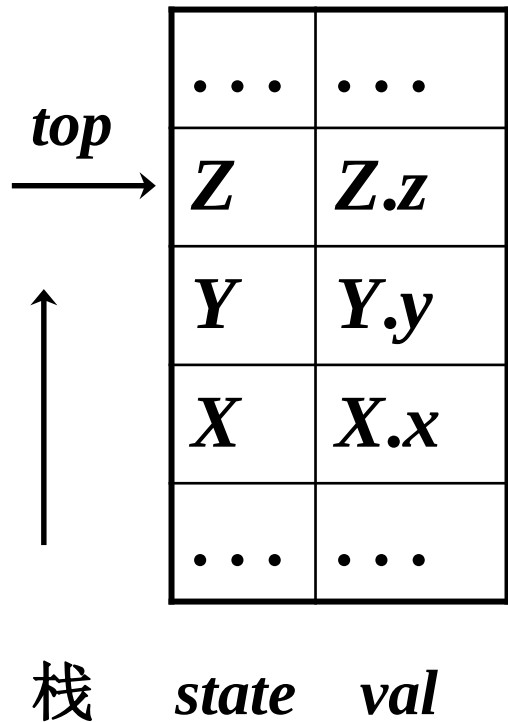
产生式	语义规则
$L \rightarrow E\ n$	$print(E.val)$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

参见: bison-examples.tar.gz 中的config/expr1.y, expr.lex



自下而上的翻译

例 简单计算器的语法制导定义改成栈操作代码



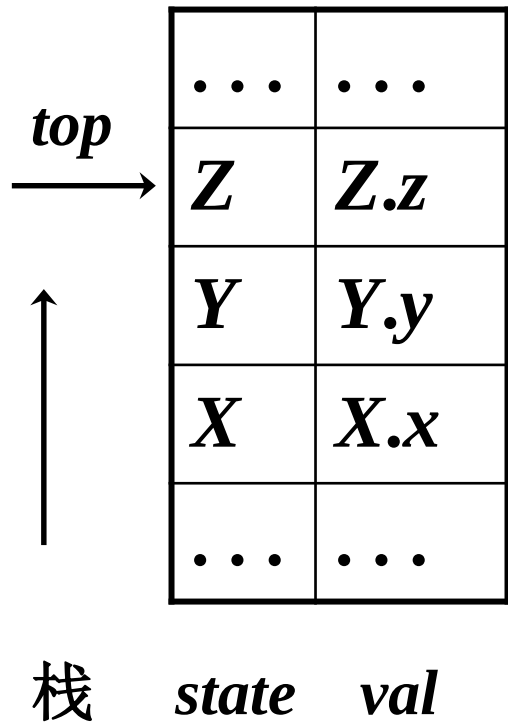
产生式	代码段
$L \rightarrow E n$	$print(val[top-1])$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

注：栈顶位置指示器 top 的修改由原来的解析程序在语义动作执行后去做



自下而上的翻译

例 简单计算器的语法制导定义改成栈操作代码



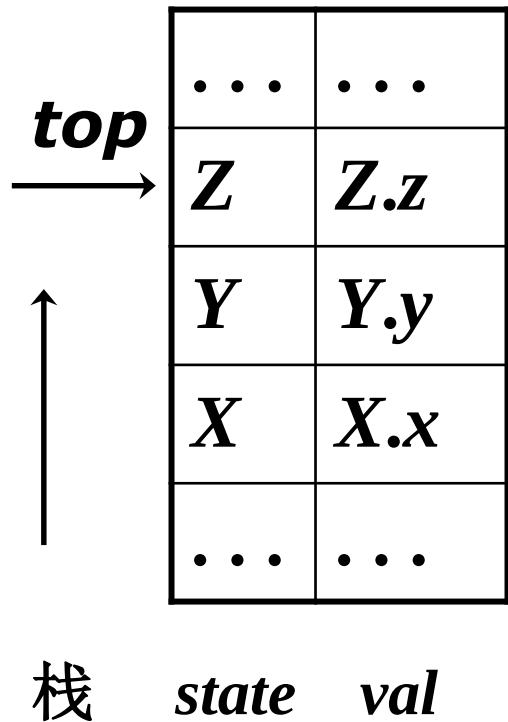
产生式	代码段
$L \rightarrow E n$	<code>print (val[top-1]);</code>
$E \rightarrow E_1 + T$	<code>val[top-2] = val [top-2] +val[top];</code>
$E \rightarrow T$	<code>$E.val = T.val$</code>
$T \rightarrow T_1 * F$	<code>$T.val = T_1.val * F.val$</code>
$T \rightarrow F$	<code>$T.val = F.val$</code>
$F \rightarrow (E)$	<code>$F.val = E.val$</code>
$F \rightarrow \text{digit}$	<code>$F.val = \text{digit.lexval}$</code>

注：栈顶位置指示器 top 的修改由原来的解析程序在语义动作执行后去做



自下而上的翻译

例 简单计算器的语法制导定义改成栈操作代码



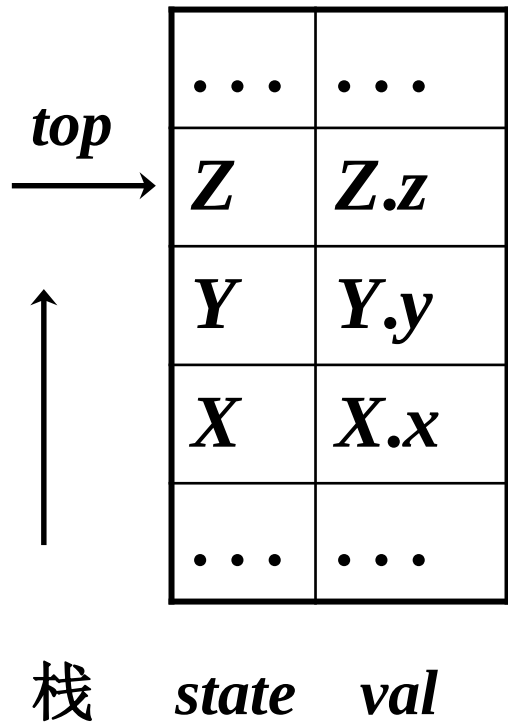
产生式	代码段
$L \rightarrow E n$	$print(val[top-1]);$
$E \rightarrow E_1 + T$	$val[top-2] = val[top-2] + val[top];$
$E \rightarrow T$	
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

注：栈顶位置指示器 top 的修改由原来的解析程序在语义动作执行后去做



自下而上的翻译

例 简单计算器的语法制导定义改成栈操作代码

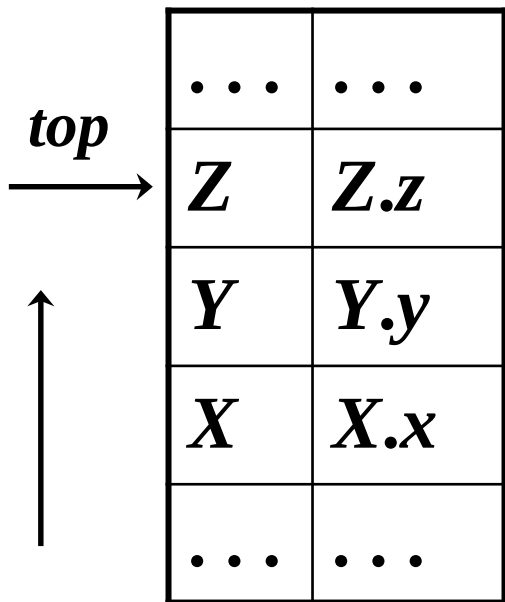


产生式	代码段
$L \rightarrow E n$	<code>print (val[top-1]);</code>
$E \rightarrow E_1 + T$	<code>val[top-2] = val [top-2] +val[top];</code>
$E \rightarrow T$	
$T \rightarrow T_1 * F$	<code>val[top-2] = val [top-2] × val[top];</code>
$T \rightarrow F$	<code><i>T.val = F.val</i></code>
$F \rightarrow (E)$	<code><i>F.val = E.val</i></code>
$F \rightarrow \text{digit}$	<code><i>F.val = digit.lexval</i></code>

注：栈顶位置指示器 top 的修改由原来的解析程序在语义动作执行后去做



例 简单计算器的语法制导定义改成栈操作代码



栈 *state val*

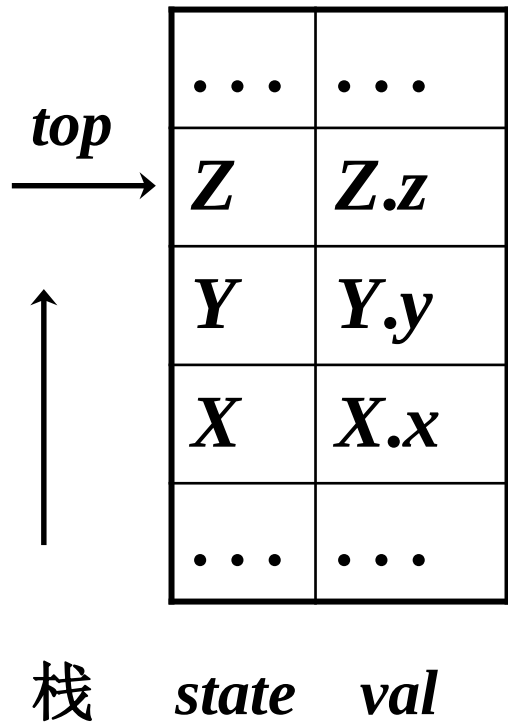
产生式	代码段
$L \rightarrow E n$	<i>print</i> (<i>val[top-1]</i>);
$E \rightarrow E_1 + T$	<i>val[top-2] = val [top-2] +val[top];</i>
$E \rightarrow T$	
$T \rightarrow T_1 * F$	<i>val[top-2] = val [top-2] × val[top];</i>
$T \rightarrow F$	
$F \rightarrow (E)$	<i>F.val = E.val</i>
$F \rightarrow \text{digit}$	<i>F.val = digit.lexval</i>

注：栈顶位置指示器top的修改由原来的解析程序在语义动作执行后去做



自下而上的翻译

例 简单计算器的语法制导定义改成栈操作代码



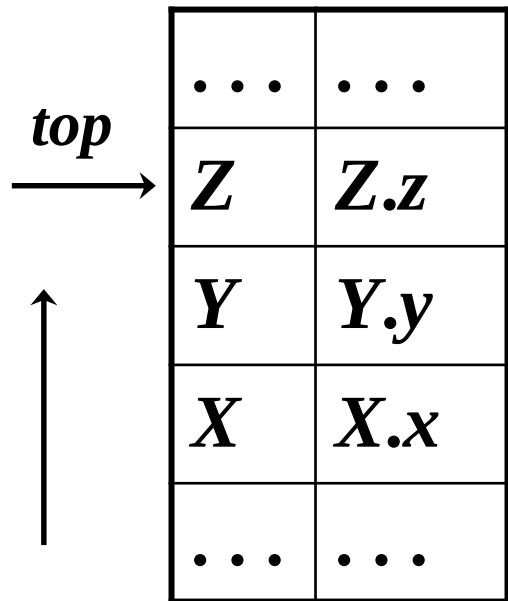
产生式	代码段
$L \rightarrow E n$	$print(val[top-1]);$
$E \rightarrow E_1 + T$	$val[top-2] = val[top-2] + val[top];$
$E \rightarrow T$	
$T \rightarrow T_1 * F$	$val[top-2] = val[top-2] \times val[top];$
$T \rightarrow F$	
$F \rightarrow (E)$	$val[top-2] = val[top-1];$
$F \rightarrow digit$	$F.val = digit.lexval$

注：栈顶位置指示器 top 的修改由原来的解析程序在语义动作执行后去做



自下而上的翻译

例 简单计算器的语法制导定义改成栈操作代码



栈 *state* *val*

产生式	代码段
$L \rightarrow E n$	$print(val[top-1]);$
$E \rightarrow E_1 + T$	$val[top-2] = val[top-2] + val[top];$
$E \rightarrow T$	
$T \rightarrow T_1 * F$	$val[top-2] = val[top-2] \times val[top];$
$T \rightarrow F$	
$F \rightarrow (E)$	$val[top-2] = val[top-1];$
$F \rightarrow digit$	

注：栈顶位置指示器 top 的修改由原来的解析程序在语义动作执行后去做



Bison举例bison-examples: config/expr.y

```
%{  
#include <stdio.h>  
#include <math.h>  
%}  
  
%union {  
    float val;  
}  
%token NUMBER  
%token PLUS MINUS MULT DIV EXPON  
...  
%left MINUS PLUS  
%left MULT DIV  
%right EXPON  
...  
%type <val> exp NUMBER  
%%
```

各种语义值类型和域
置于共用体中

声明exp、Number
的语义值是在val域

```
input : | input line  
      ;  
  
...  
exp  : NUMBER          { $$ = $1; }  
    | exp PLUS exp      { $$ = $1 + $3; }  
    | exp MINUS exp      { $$ = $1 - $3; }  
    | exp MULT exp       { $$ = $1 * $3; }  
    | exp DIV exp        { $$ = $1 / $3; }  
    | MINUS exp %prec MINUS { $$ = -$2; }  
    | exp EXPON exp       { $$ = pow($1,$3); }  
    | LB exp RB           { $$ = $2; }  
    ;  
  
%%  
yyerror(char *message)  
{ printf("%s\n",message);}  
int main(int argc, char *argv[])  
{ yyparse(); return(0);}
```



例题 2

为下面文法写一个语法制导的定义，用S的综合属性 val 给出下面文法中S产生的二进制数的值。

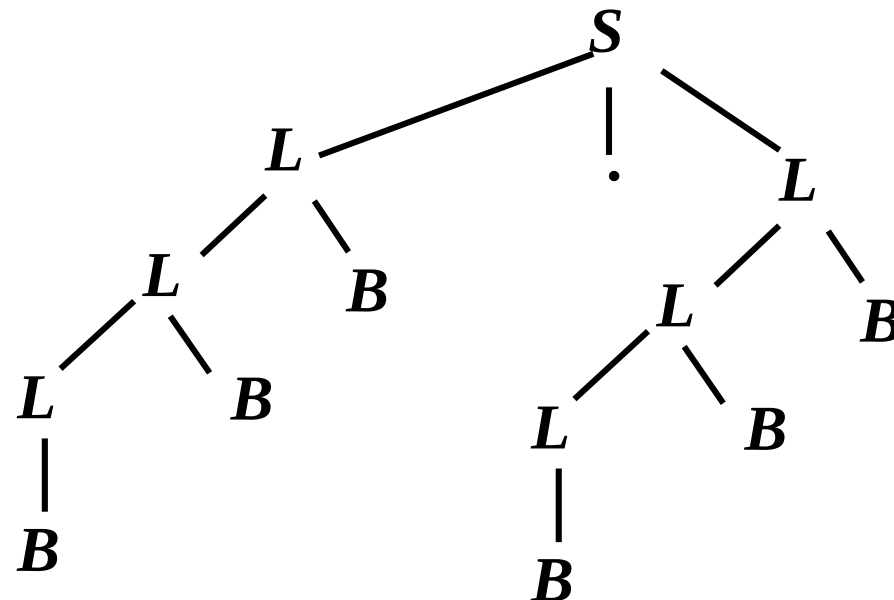
例如，输入101.101时， $S.val = 5.625$ （可以修改文法）

若按 $2^2 + 0 + 2^0 + 2^{-1} + 0 + 2^{-3}$ 来计算，该文法对小数点左边部分的计算不利，因为需要继承属性来确定每个B离开小数点的距离

$$S \rightarrow L . L \mid L$$

$$L \rightarrow L B \mid B$$

$$B \rightarrow 0 \mid 1$$





例题 2

为下面文法写一个语法制导的定义，用S的综合属性 val 给出下面文法中S产生的二进制数的值。

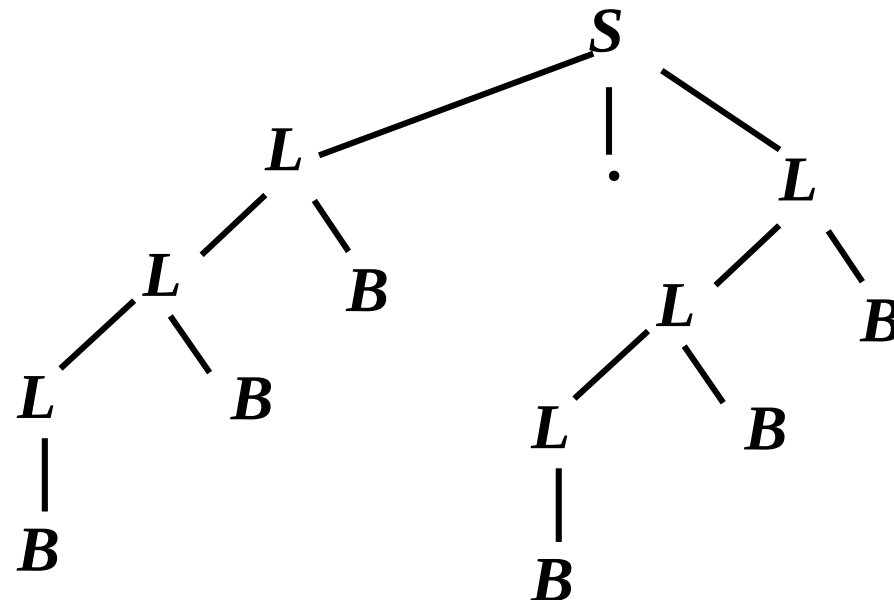
例如，输入101.101时， $S.val = 5.625$ （可以修改文法）

若小数点左边按 $(1 \times 2 + 0) \times 2 + 1$ 计算。该办法不能直接用于小数点右边，需改成 $((1 \times 2 + 0) \times 2 + 1)/2^3$ ，这时需要综合属性来统计B的个数

$$S \rightarrow L . L \mid L$$

$$L \rightarrow L B \mid B$$

$$B \rightarrow 0 \mid 1$$





例题 2

为下面文法写一个语法制导的定义，用S的综合属性 val 给出下面文法中S产生的二进制数的值。

例如，输入101.101时， $S.val = 5.625$ （可以修改文法）

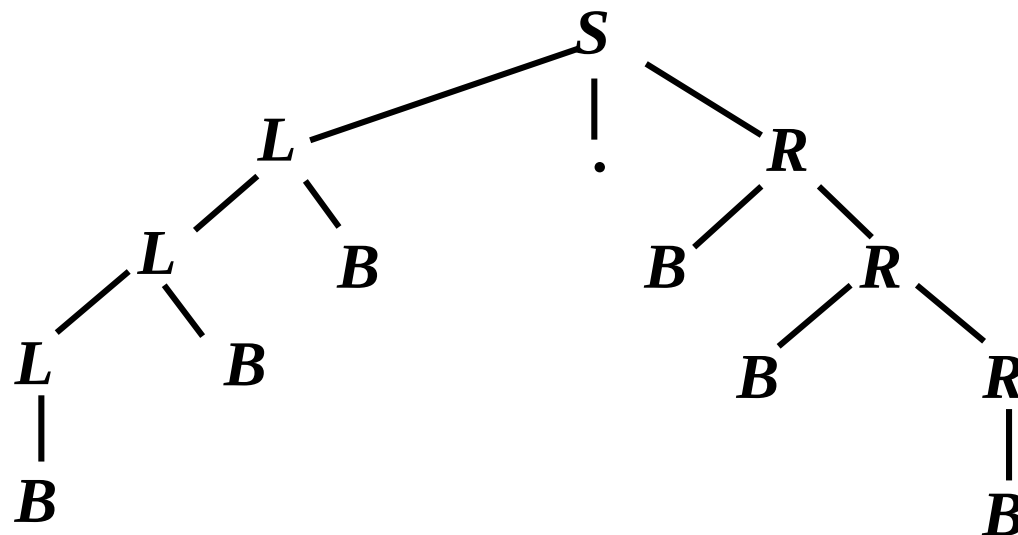
更清楚的办法是将文法改成下面的形式

$S \rightarrow L . R \mid L$

$L \rightarrow L B \mid B$

$R \rightarrow B R \mid B$

$B \rightarrow 0 \mid 1$





例题 2

为下面文法写一个语法制导的定义，用S的综合属性*val* 给出下面文法中S产生的二进制数的值。

例如，输入101.101时， $S.val = 5.625$ （可以修改文法）

$S \rightarrow L . R$ $S.val = L.val + R.val$

$S \rightarrow L$ $S.val = L.val$

$L \rightarrow L_1 B$ $L.val = L_1.val \times 2 + B.val$

$L \rightarrow B$ $L.val = B.val$

$R \rightarrow B R_1$ $R.val = R_1.val / 2 + B.val / 2$

$R \rightarrow B$ $R.val = B.val / 2$

$B \rightarrow 0$ $B.val = 0$

$B \rightarrow 1$ $B.val = 1$



例题 3

给出把中缀表达式翻译成没有冗余括号的中缀表达式的语法制导定义。例如，因为+和*是左结合，

$((a * (b + c)) * (d))$ 可以重写成 $a * (b + c) * d$

两种方法：

- 先把括号都去掉，然后在必要的地方再加括号
- 去掉表达式中的冗余括号，保留必要的括号



例题 3

□ 先把括号都去掉，然后在必要的地方再加括号

$S' \rightarrow E \quad \text{print} (E. \text{code})$

$E \rightarrow E_1 + T$

if $T. \text{op} == \text{plus}$ then

$E. \text{code} = E_1. \text{code} || \text{"+"} || \text{"("} || T. \text{code} || \text{"})"$

else

$E. \text{code} = E_1. \text{code} || \text{"+"} || T. \text{code};$

$E. \text{op} = \text{plus}$

$E \rightarrow T \quad E. \text{code} = T. \text{code}; E. \text{op} = T. \text{op}$



例题 3

□ 先把括号都去掉，然后在必要的地方再加括号

$T \rightarrow T_1 * F$

if ($F.op == plus$) or ($F.op == times$) then

if $T_1.op == plus$ then

$T.code = "(" \parallel T_1.code \parallel ")" \parallel "*" \parallel "(" \parallel F.code \parallel ")"$

else

$T.code = T_1.code \parallel "*" \parallel "(" \parallel F.code \parallel ")"$

else if $T_1.op = plus$ then

$T.code = "(" \parallel T_1.code \parallel ")" \parallel "*" \parallel F.code$

else

$T.code = T_1.code \parallel "*" \parallel F.code;$

$T.op = times$



例题 3

□ 先把括号都去掉，然后在必要的地方再加括号

$T \rightarrow F$

$T.code = F.code; T.op = F.op$

$F \rightarrow id$

$F.code = id.lexeme; F.op = id$

$F \rightarrow (E)$

$F.code = E.code; F.op = E.op$



例题 3

□ 去掉表达式中的冗余括号，保留必要的括号

- 给 E ， T 和 F 两个继承属性 $left_op$ 和 $right_op$ 分别表示左右两侧算符的优先级
- 给它们一个综合属性 $self_op$ 表示自身主算符的优先级
- 再给一个综合属性 $code$ 表示没有冗余括号的代码
- 分别用1和2表示加和乘的优先级，用3表示 id 和 (E) 的优先级，用0表示左侧或右侧没有运算对象的情况



例题 3

$S' \rightarrow E$

$E. left_op = 0; E. right_op = 0; print (E. code)$

$E \rightarrow E_1 + T$

$E_1. left_op = E. left_op; E_1. right_op = 1;$

$T. left_op = 1; T. right_op = E. right_op;$

$E. code = E_1. code || "+" || T. code ; E. self_op = 1;$

$E \rightarrow T$

$T. left_op = E. left_op;$

$T. right_op = E. right_op;$

$E. code = T. code; E. self_op = T. self_op$



例题 3

$T \rightarrow T_1 * F \quad \dots$

$T \rightarrow F \quad \dots$

$F \rightarrow \text{id}$

$F.\text{code} = \text{id}.\text{lexeme}; F.\text{self_op} = 3$



例题 3

$F \rightarrow (E)$

$E. left_op = 0; E. right_op = 0;$

$F. self_op =$ if $(F. left_op < E. self_op)$ and $(E. self_op \geq F. right_op)$
then $E. self_op$ else 3

$F. code =$ if $(F. left_op < E. self_op)$ and $(E. self_op \geq F. right_op)$
then $E. code$ else “(” || $E. code$ || “)”



中国科学技术大学
University of Science and Technology of China

下期预告： L属性定义的语义计算

- L属性定义的自上而下计算
- L属性定义的自下而上计算